Git Merge Conflict Simulation

Follow these commands to simulate a conflict between two branches and resolve it.

1. Initial Setup & Remote Connection

```
Bash

mkdir git-conflict-demo && cd git-conflict-demo # Create and enter a new project folder
git init # Initialize a new local Git repository
echo "Initial Content" > system.txt # Create a base file for the system
git add system.txt # Stage the file to track changes
git commit -m "Initial commit" # Create the first snapshot in history
# Replace <URL> with your actual GitHub repo link
git remote add origin <URL> # Link your local repo to a remote GitHub repo
git push -u origin main # Push the local main branch to GitHub
```

---

2. Creating the First Divergent Path (Branch Alpha)

```
Bash

git checkout -b alpha # Create and switch to a new branch named 'alpha'
echo "Update from Alpha" > system.txt # Overwrite the file with Alpha's version
git commit -am "Alpha updated the system" # Stage and commit the change in one step
git push origin alpha # Upload Alpha's branch to GitHub
```

---

3. Creating the Second Divergent Path (Branch Beta)

```
Bash

git checkout main # Move back to the 'main' branch to start from the original state
git checkout -b beta # Create and switch to 'beta' branch (it doesn't have Alpha's code)
echo "Update from Beta" > system.txt # Overwrite the SAME line with Beta's version
git commit -am "Beta updated the system" # Commit Beta's unique version of the truth
```

---

4. Triggering the Conflict

```
Bash

# We are currently on 'beta'. We will now try to pull Alpha's work into Beta.
git merge alpha # Attempt to merge Alpha into Beta; this will trigger the conflict
```

---

5. Resolving the Conflict

At this stage, Git pauses. Use `cat system.txt` to see the conflict markers.

```
Bash

# Manually open 'system.txt' in an editor and remove the <<<<, ====, >>>> markers
# Keep the text you want (e.g., "Update from Alpha and Beta")
git add system.txt # Inform Git that you have manually fixed the collision
git commit -m "Resolved conflict between Alpha and Beta" # Finalize the merge commit
```

---

6. Final Sync to GitHub

```
Bash

git push origin beta # Push the resolved Beta branch to GitHub
git checkout main # Switch back to the primary production branch
git merge beta # Fast-forward 'main' to include the resolved work
git push origin main # Update the final version on GitHub
```

---

7. Verification

```
Bash

git log --graph --oneline --all # View the system-level "map" of your branches and merge
```

Pull Request (PR):

==========
STEP 1: CREATE REPOSITORY ON GITHUB
==========

1. Go to https://github.com
2. Click "New Repository"
3. Enter repo name (e.g., myproject)
4. Click "Create Repository"

==========
STEP 2: CONNECT LOCAL PROJECT TO GITHUB
==========

```
mkdir myproject
cd myproject

git init

echo "Hello World" > file.txt

git add .
git commit -m "Initial commit"

git remote add origin https://github.com/your-username/myproject.git

git branch -M main
git push -u origin main
```

==========
STEP 3: CREATE A NEW BRANCH

```
git checkout -b feature1
```

==========
STEP 4: MAKE CHANGES IN BRANCH
==========

```
echo "New feature added" >> file.txt
```

```
git add .
git commit -m "Added new feature"

============================================================
STEP 5: PUSH BRANCH TO GITHUB
============================================================

git push origin feature1


============================================================
STEP 6: CREATE PULL REQUEST (PR)
============================================================

1. Open your GitHub repository
2. You will see "Compare & pull request" → click it
   (OR go to "Pull Requests" tab → click "New Pull Request")

3. Select:
   Base branch: main
   Compare branch: feature1

4. Click "Create Pull Request"

5. Add title & description

6. Click "Create Pull Request"

7. Click "Merge Pull Request"

8. Click "Confirm Merge"


============================================================
STEP 7: UPDATE LOCAL MAIN (AFTER MERGE)
============================================================

git checkout main
git pull origin main
```